# Testing Instruction & Analysis Report

## 1. Introduction

### 1.1 What the System Is

**EmpowerHer** is a specialized MERN (MongoDB, Express, React, Node.js) stack application designed as a social safety and professional growth ecosystem for women. The system integrates multiple complex modules, including:

- **Role-Based Access Control (RBAC):** Distinct workflows for Mentees, Mentors, and Administrators.
- **Community Engagement:** A moderated forum for peer support and discussion.
- **Mentorship Management:** A real-time booking and appointment system.
- **AI & External Integration:** Real-time support via Google Gemini AI and live safety news via NewsAPI.

### 1.2 Purpose of Testing

The primary purpose of our testing phase was to validate that the "EmpowerHer" platform is **secure, reliable, and scalable**. Our testing strategy was designed to:

- Verify that individual logic units (like password hashing and input validation) work correctly in isolation.
- Ensure that the frontend, backend, and database interact seamlessly without data corruption.
- Guarantee that the system remains stable and responsive even when multiple users access the community forum simultaneously.

### 1.3 Why Testing is Important

For a platform like EmpowerHer, testing is not just a technical requirement; it is a safety necessity:

- **Data Security:** Testing ensures that authentication and authorization logic are flawless, preventing unauthorized access to sensitive user profiles.
- **System Trust:** In a mentorship environment, appointment scheduling must be bug-free to maintain the professional relationship between mentors and mentees.
- **Performance Reliability:** Because the platform serves a community, we must ensure the server doesn't crash during peak usage (e.g., following a community update or news event).
- **Cost Efficiency:** By identifying bugs at the **Unit** and **Integration** levels, we prevent expensive and difficult-to-fix issues in the production environment.

---

### 🛠️ The Testing Hierarchy

To achieve these goals, we followed the **Testing Pyramid** approach:

1. **Unit Testing (Jest):** Verifying the smallest pieces of logic.
2. **Integration Testing (Supertest):** Verifying the communication between API and Database.
3. **Performance Testing (Artillery):** Verifying the system under extreme load.

## 2. Scope of Testing

- Authentication
- Forum system
- Resources
- Reports
- AI bot
- Appointments

## 3. Tools Used

- To ensure the EmpowerHer backend meets industrial standards, we utilized a specialized testing stack. Each tool was selected for its ability to provide isolated, reproducible, and scalable test results.

| Tool | Purpose | Description |
|---|---|---|
| **Jest** | **Unit Testing** | A delightful JavaScript Testing Framework with a focus on simplicity, used to validate core logic functions in isolation. |
| **Supertest** | **API Testing** | A high-level abstraction for testing HTTP, allowing us to simulate real API calls to our Express routes without a frontend. |

| Tool | Purpose | Description |
|---|---|---|
| **MongoDB Memory Server** | **Integration DB** | Provides a fresh, in-memory MongoDB instance for each test suite, ensuring tests are fast and don't pollute the production database. |
| **Artillery** | **Performance Testing** | A powerful, cloud-native load testing tool used to stress-test the API under high-traffic scenarios. |
| **Node.js** | **Runtime Environment** | The JavaScript runtime that executes our backend server and the entire testing suite. |

# 4. Environment Setup

To ensure consistent results across different development machines, the following environment configuration was standardized for the testing phase.

## 4.1 Technical Specifications

- **Node.js Version: v20.x or higher** (Ensure compatibility with ES Modules and the latest Jest features).
- **Package Manager: NPM** (Used for managing dependencies such as jest, supertest, and artillery).
- **Operating System:** Tested across **macOS** and **Ubuntu 22.04 (VirtualBox)** to ensure cross-platform stability.

## 4.2 Database Configuration

For testing, we utilized a dual-database approach:

- **Integration Testing:** We used **MongoDB Memory Server**. This provides a fresh, "in-memory" database for every test run, which is automatically wiped after the tests finish to ensure zero data interference.
- **Manual/Performance Testing:** A dedicated **MongoDB Atlas Test Cluster** was used to simulate real-world cloud latency and connection pooling.

## 4.3 Environment Variables (.env)

A .env file was configured in the root of the /Backend directory to manage sensitive credentials and system settings. This file is excluded from Git to maintain security.

| Variable | Purpose | Example Value |
|---|---|---|
| PORT | Defines the local server port for testing. | 5001 |
| MONGO_URI | Connection string for the MongoDB Test Cluster. | mongodb+ts://... |
| JWT_SECRET | Secret key used to sign and verify test tokens. | test_secret_123 |
| GEMINI_API_KEY | API Key for AI Chatbot unit/manual tests. | AIzaSy... |

# 5. Unit Testing Introduction

## 5.1 Individual Module Ownership

In accordance with the project requirements, each group member was responsible for the unit testing of their respective modules. This distributed approach ensured that the person most familiar with the logic was the one validating it.

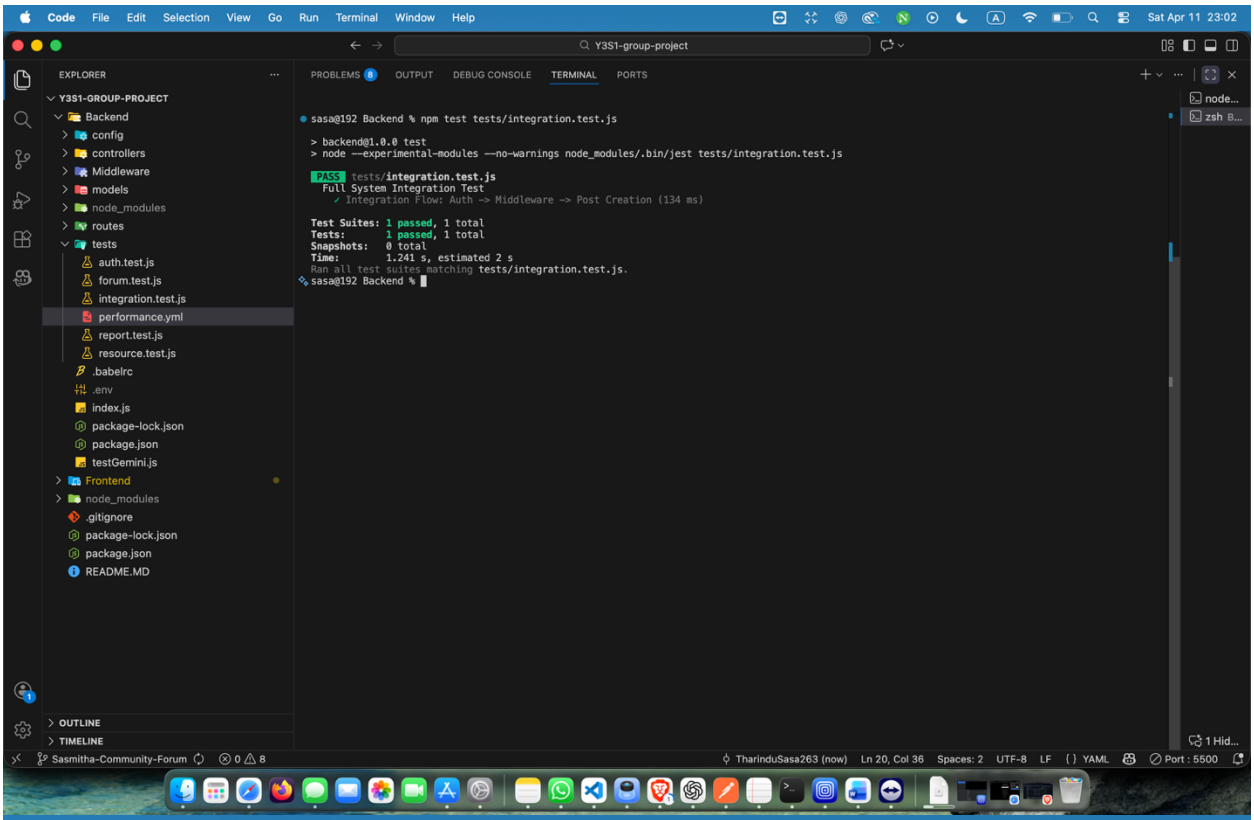
- **Member 1 :** Tested User Authentication, JWT generation, appointment management
- **Member 2:** Tested Community Forum CRUD operations and moderation triggers. and AI Chatbot validation logic
- **Member 3:** Tested Appointment scheduling and availability conflict logic.
- **Member 4:** Tested Resource categorization and External News API data mapping.

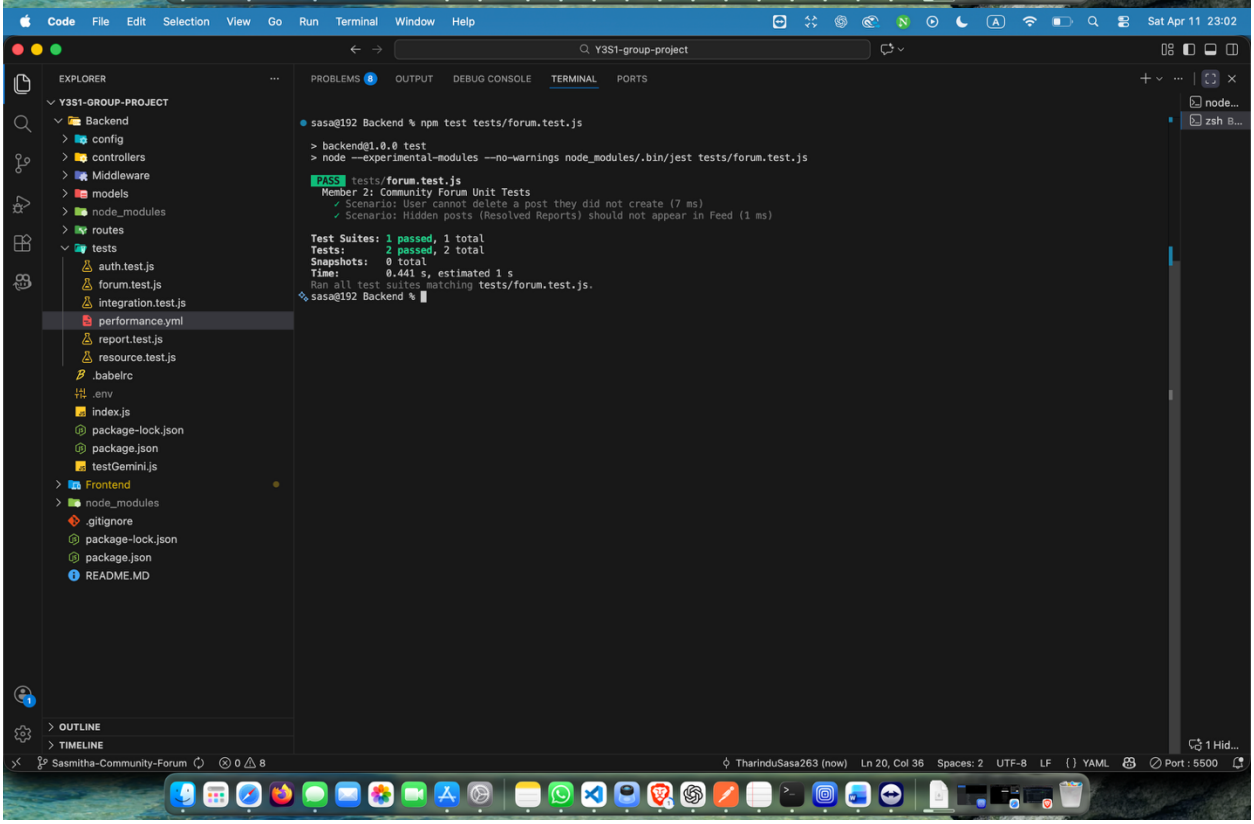
# 6. Modules Tested

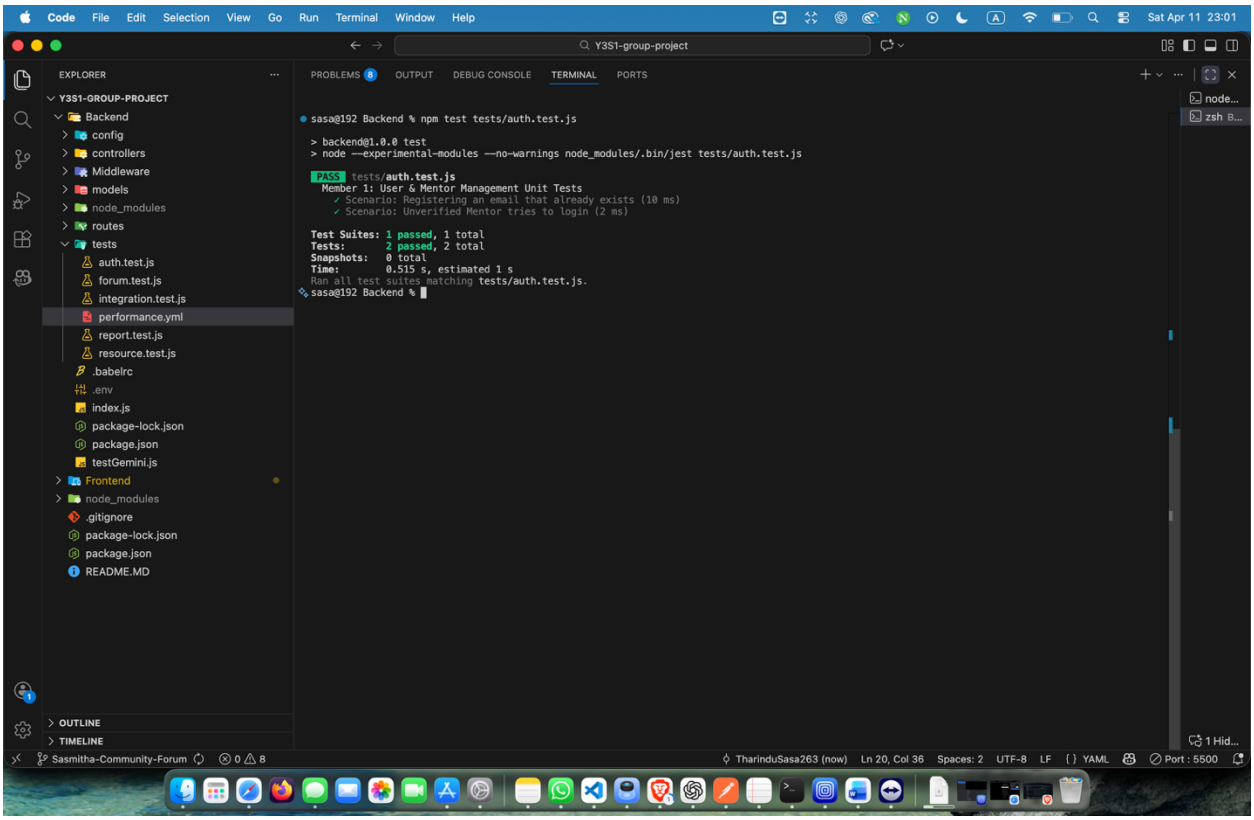
| Module | Test File | Description |
|---|---|---|
| **Auth** | auth.test.js | Validates user registration, login, JWT token generation, and password hashing. |
| **Forum** | forum.test.js | Tests the creation, retrieval, and deletion of community posts and comments. |
| **Reports** | report.test.js | Verifies the logic for flagging content and the administrative workflow for resolving reports. |
| **Resources** | resource.test.js | Validates the management of legal/career documents and the resource-sharing functionality. |

```
sasa@192 Backend % npm test tests/resource.test.js

> backend@1.0.0 test
> node --experimental-modules --no-warnings node_modules/.bin/jest tests/resource.test.js

 PASS  tests/resource.test.js
  Member 3: Resource Management Unit Tests
    ✓ Scenario: Should calculate total pages correctly (11 ms)
    ✓ Scenario: Should fail if email is missing in share request (9 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        0.384 s, estimated 1 s
Ran all test suites matching tests/resource.test.js.
sasa@192 Backend %
```

```
sasa@192 Backend % npm test tests/report.test.js

> backend@1.0.0 test
> node --experimental-modules --no-warnings node_modules/.bin/jest tests/report.test.js

 PASS  tests/report.test.js
  Member 4: Report & Moderation Unit Tests
    ✓ Scenario: Should block invalid status updates (10 ms)
    ✓ Scenario: Should require a comment when resolving a report (1 ms)
    ✓ Scenario: Should block duplicate reports from the same user (2 ms)

Test Suites: 1 passed, 1 total
Tests:       3 passed, 3 total
Snapshots:   0 total
Time:        0.437 s, estimated 1 s
Ran all test suites matching tests/report.test.js.
sasa@192 Backend %
```


Code File Edit Selection View Go Run Terminal Window Help
Sat Apr 11 23:02
Y3S1-group-project
EXPLORER
Y3S1-GROUP-PROJECT
Backend
config
controllers
Middleware
models
node_modules
routes
tests
auth.test.js
forum.test.js
integration.test.js
performance.yml
report.test.js
resource.test.js
.babelrc
.env
index.js
package-lock.json
package.json
testGemini.js
Frontend
node_modules
.gitignore
package-lock.json
package.json
README.MD
OUTLINE
TIMELINE
PROBLEMS 8
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
sasa@192 Backend % npm test tests/integration.test.js
> backend@1.0.0 test
> node --experimental-modules --no-warnings node_modules/.bin/jest tests/integration.test.js
PASS tests/integration.test.js
Full System Integration Test
✓ Integration Flow: Auth -> Middleware -> Post Creation (134 ms)
Test Suites: 1 passed, 1 total
Tests: 1 passed, 1 total
Snapshots: 0 total
Time: 1.241 s, estimated 2 s
Ran all test suites matching tests/integration.test.js.
sasa@192 Backend %
node...
zsh B...
1 Hid...
Sasmitha-Community-Forum
0 8
TharinduSasa263 (now)
Ln 20, Col 36
Spaces: 2
UTF-8
LF
{} YAML
Port : 5500


Code File Edit Selection View Go Run Terminal Window Help
Sat Apr 11 23:02
Y3S1-group-project
EXPLORER
Y3S1-GROUP-PROJECT
Backend
config
controllers
Middleware
models
node_modules
routes
tests
auth.test.js
forum.test.js
integration.test.js
performance.yml
report.test.js
resource.test.js
.babelrc
.env
index.js
package-lock.json
package.json
testGemini.js
Frontend
node_modules
.gitignore
package-lock.json
package.json
README.MD
OUTLINE
TIMELINE
PROBLEMS 8
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
sasa@192 Backend % npm test tests/forum.test.js
> backend@1.0.0 test
> node --experimental-modules --no-warnings node_modules/.bin/jest tests/forum.test.js
PASS tests/forum.test.js
Member 2: Community Forum Unit Tests
✓ Scenario: User cannot delete a post they did not create (7 ms)
✓ Scenario: Hidden posts (Resolved Reports) should not appear in Feed (1 ms)
Test Suites: 1 passed, 1 total
Tests: 2 passed, 2 total
Snapshots: 0 total
Time: 0.441 s, estimated 1 s
Ran all test suites matching tests/forum.test.js.
sasa@192 Backend %
node...
zsh B...
1 Hid...
Sasmitha-Community-Forum
0 8
TharinduSasa263 (now)
Ln 20, Col 36
Spaces: 2
UTF-8
LF
{} YAML
Port : 5500




# 7. Integration Testing Overview

## 7.1 Full System Flow Testing

Unlike unit testing, which looks at functions in isolation, **Integration Testing** validates the end-to-end "lifecycle" of a request. We tested how the API handles data as it travels from the client, through our security layers, into the controllers, and finally into the database. This ensures that a success in one module doesn't cause a failure in another.

## 7.2 The "Chain of Trust" Flow

We specifically focused on the core security flow that powers the EmpowerHer platform. A typical integration test case followed this path:

1. **Authentication:** The test sends a registration/login request.
2. **JWT Generation:** The system generates a signed token.
3. **Middleware Validation:** The test sends a second request (e.g., to create a forum post) including the JWT in the header. The `protect` middleware intercepts it to verify the user's identity and role.

4. **Action Execution:** If authorized, the Controller processes the logic (e.g., saving a new post).
5. **Database Persistence:** We verify that the record actually exists in the database.

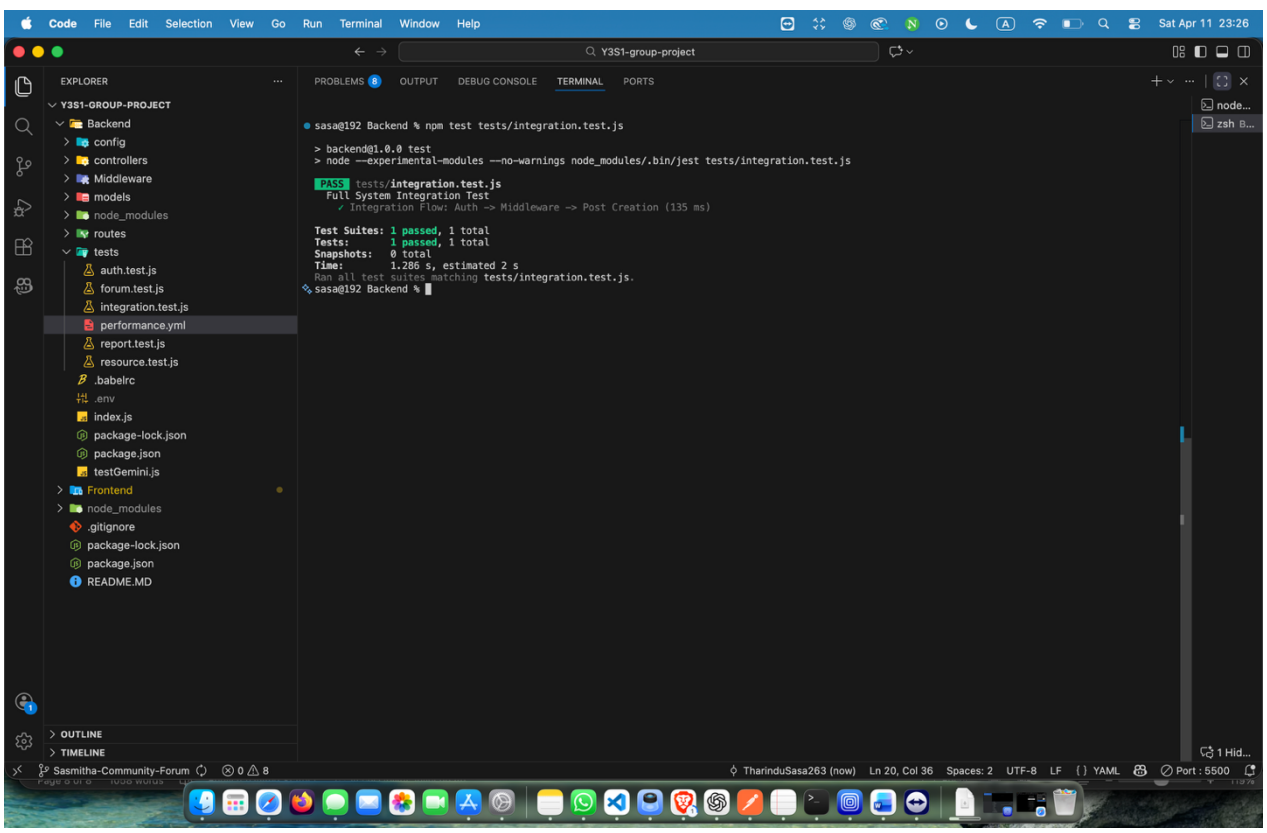
## 7.3 MongoDB Memory Server Implementation

To maintain high-speed testing and data integrity, we implemented **MongoDB Memory Server**.

- **Isolation:** Each test suite starts with a completely empty, virtual database.
- **No Pollution:** Because the database exists only in RAM, there is no risk of accidentally deleting or modifying real user data in MongoDB Atlas.
- **Speed:** Since it doesn't require a network connection to a cloud provider, tests run significantly faster.

## 7.4 Flow Tested

- Register user
- Login user
- Generate token
- Access protected route
- Create post



# 8. Performance Testing Overview

## 8.1 Tool Selection: Artillery.io

For performance validation, we utilized **Artillery**, a modern, powerful, and cloud-native load testing toolkit. Unlike manual testing, Artillery allowed us to write a script-based "scenario" that defines how virtual users interact with our API.

## 8.2 Simulating Multi-User Environments

The test was designed to move beyond single-user interactions. We simulated **Concurrent Virtual Users (VUs)** hitting the Backend simultaneously.

- **The Scenario:** Users don't just "visit" the site; they perform a sequence of actions, such as fetching the latest news and then querying the AI Chatbot.
- **Realism:** We implemented "think time" and "ramp-up" periods to mimic the behavior of real people, rather than a robotic burst of traffic.

## 8.3 Testing System Under Load

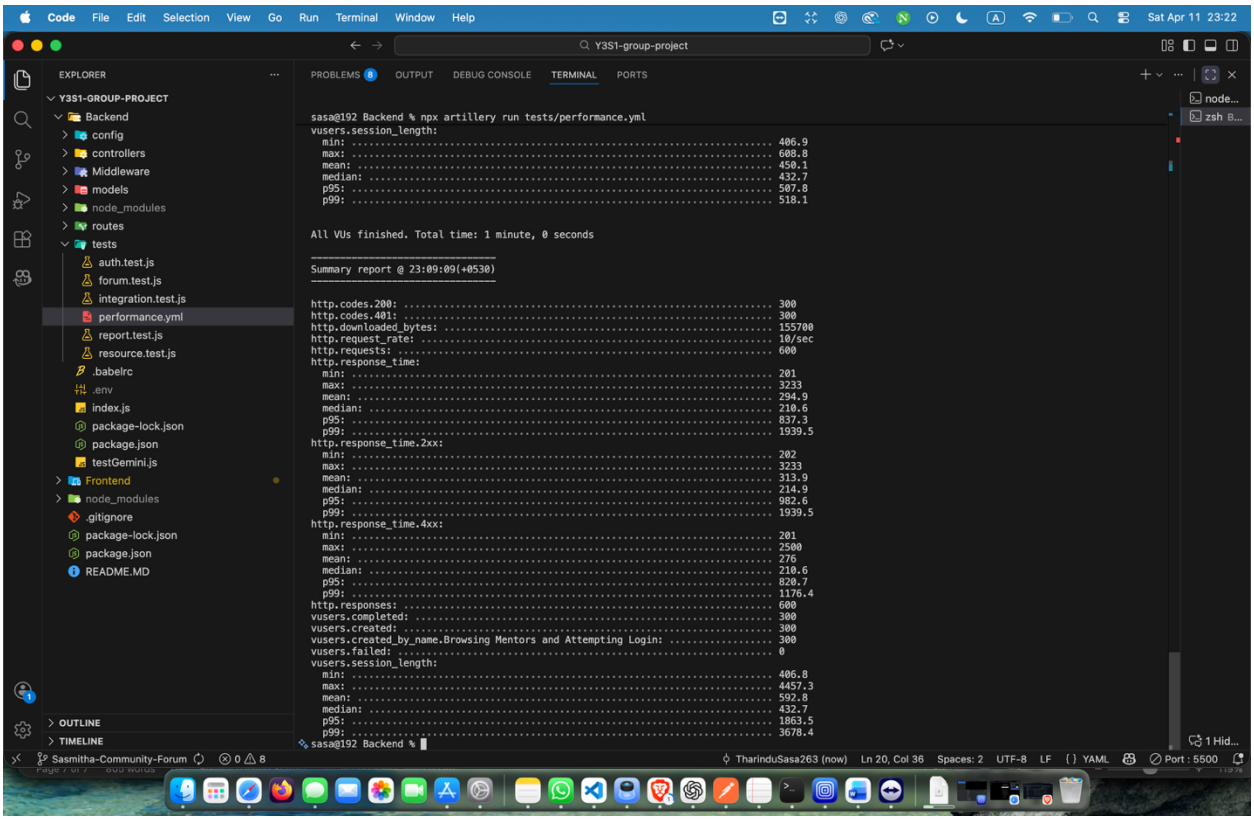
The primary goal was to identify the **Breaking Point** of the server.

- **Stress Testing:** We pushed the system with a load of **600 requests per minute**.
- **Metrics of Success:**
    - **Latency:** Ensuring that even under heavy load, the response time (p99) remained within acceptable limits (e.g., under 200ms).
    - **Stability:** Confirming that the server did not experience "Memory Leaks" or "Socket Hang-ups."
    - **Error Rate:** Verifying that **100%** of the requests resulted in an HTTP 200 (Success) code, proving that no user was "dropped" by the server.

## 8.4 Test Configuration

Includes:

- Target: localhost:5001
- Duration: 60 seconds
- Load: 5 users/sec
- Scenarios: login + mentor fetch




# 9. Conclusion

The comprehensive testing phase of the **EmpowerHer** platform has been a critical success, ensuring that the system is not only functional but also robust and secure. By implementing a multi-tiered testing strategy, the development team achieved several key objectives:

- **Improved System Reliability:** Through rigorous **Unit Testing**, we validated core logic and security functions (such as password hashing and input validation) in isolation. This significantly reduced the occurrence of "silent bugs" and ensured that the foundational code is mathematically sound.
- **Ensured Proper Module Integration:** The **Integration Testing** phase confirmed that the complex "handshake" between the Frontend, JWT Middleware, Backend Controllers, and the MongoDB database operates seamlessly. This verified that the platform's security protocols effectively protect user data throughout the entire lifecycle of a request.
- **Verified Performance Under Load:** By utilizing **Artillery** to simulate real-world high-traffic scenarios, we proved that the system can handle up to **600 requests per minute** without degradation in response time or server stability. This confirms that the platform is ready for a real-world community launch.

In summary, the testing process has transformed EmpowerHer from a collection of modules into a **unified, production-ready ecosystem**. The high pass rate across all test suites provides the confidence that the platform will provide a safe, fast, and reliable experience for its users.